

Software Architecture
(T630019402)

Lecture #4, March 3, 2026

Jukka Ruohonen

An Announcement

- ▶ The lecture on **24 March is canceled** due to a conference, and there is **no lecture on 31 March** due to the Easter break
- ▶ Therefore, some of the lectures in April are presumably (but not necessarily) more packed than initially planned

Case #3 Presentations

- ▶ Consider Fig. 4 in case-3.pdf¹
- ▶ **What kind** of an architecture you think it is in overall?
- ▶ Which symbols (A to D) might belong to a **deployment**?
- ▶ Do you recognize any **styles and patterns** used and where?
- ▶ Why might they use **multiple databases**?
- ▶ How would you **simplify** it with a different **view**?
 - A drawing would be preferable but not necessary

¹ <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10081336>

Today's Lecture

- ▶ Today we continue the previous lecture's theme by focusing on **testability** and **deployability** as distinct quality attributes
 - Testability is the "*susceptibility to testing*"², and the adjective "*susceptible*" means that something is "*capable of submitting to an action, process, or operation*"³
 - Deployability is not a well-recognized word according to online dictionaries, but we can understand it to mean that a developed system can be actually deployed for production with a *predictable and acceptable amount of time and effort* (Bass et al. 2021)

² <https://www.merriam-webster.com/dictionary/testability>

³ <https://www.merriam-webster.com/dictionary/susceptible>

Question

How do you understand testability?

Testability (1/2)

- ▶ Bass et al. (2021) characterize testability as “*the ease with which software can be made to demonstrate its faults*”
 - Note that their characterization frames testability to software **verification** and not validation of requirements
 - Know your history; it was Boehm (1984) who originally made the distinction between software validation (“*building the right product*”) and the verification (“*building the product right*”) of the software developed
 - Furthermore, their characterization assumes that a software has at least one fault, which is theoretically a problematic standpoint but still acceptable to convey the message

Testability (2/2)

- ▶ Ammann & Offutt (2008, p. 284) motivate the importance of testability with the following four points:
 1. If testability is low, developers know that testing alone will not do a good job at verifying; **something else** is needed
 - But what that something else would even be?
 2. Therefore, *“any approach that improves testability will almost certainly be **worth the investment**”*
 3. If testability is high, test results **increase confidence** about software quality
 4. *“In cases where testability is low and resistant to improvement, and where alternative means of verification are impractical, the test engineer has a solid argument to **take to management** that the artifact poses an **undue amount of risk**”*

Testability Scenarios

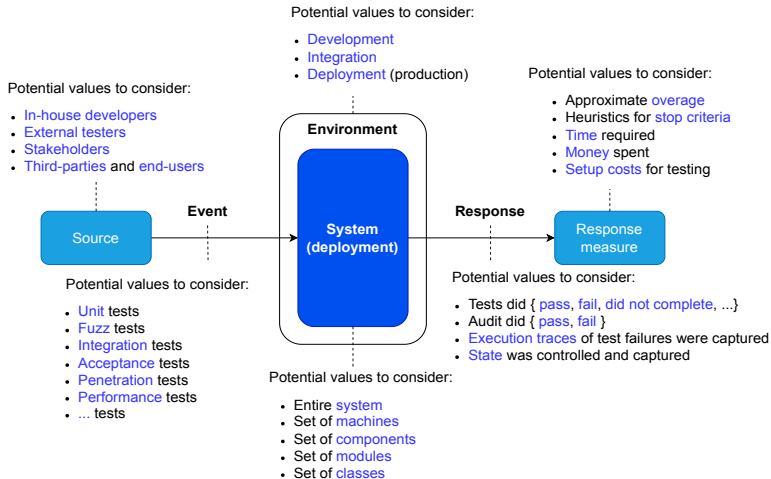


Figure 1: Adopted and modified from Bass et al. (2021)

Testing and the Separation of Concerns

have the stated properties". Business data processing systems are sufficiently complicated to require such a separation of concerns and the suggestion that in that part of the computing world "scientific thought is a non-applicable luxury" puts the cart before the horse: the mess they are in has been caused by too much unscientific thought.

ambiguous specifications." From a logical point of view, this objection is nonsense: if your specifications are contradictory, life is very easy, for then you know that no program will satisfy them, so, make "no program"; if your specifications are ambiguous, the greater the ambiguity, the easier the specifications are to satisfy (if the specifications are absolutely ambiguous, every program will satisfy them!).

Figure 2:

<https://www.cs.utexas.edu/~EWD/transcriptions/EWD04xx/EWD447.html>

Testing and Verification

the most obvious blunders will show up. The first moral of this story is that *program testing can be used very efficiently to show the presence of bugs, but never to show their absence.*

under all circumstances. This is done in order to reduce the number of test cases needed; in the case of programs —i.e. abstract mechanisms— there are now strong indications that this approach can be so effective that the number of test cases needed is eventually reduced to zero, i.e. that the correctness can be shown a priori.

Figure 3:

<https://www.cs.utexas.edu/~EWD/transcriptions/EWD02xx/EWD288.html>

On the Stop Criterion

- ▶ Even with a system with almost perfect testability, the fundamental and *unresolved* issue concerning coverage and stop criterion is present; **when to stop testing?**
- ▶ Some heuristics (e.g., via `coverage.py`) could be used for deducing an answer in practice
- ▶ The question is classical in academic research; one way to approach it is with statistical **reliability** engineering models, such as those based on the non-homogenous Poisson process
- ▶ Further note that tests cause **maintenance overhead** too

Testing (1/6)

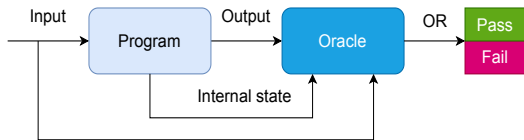


Figure 4: It Makes Sense But Why Is It Not Sufficient?

Testing (2/6)

- ▶ There are many **oracle variants** (Barr et al. 2015):
 1. **Specified oracles** are what we often think testing to be about; $1 + 1 = 2$, $f(x) > 0 \forall x$, and so forth and so on
 2. **Pseudo-oracles** are often understood as alternative versions developed independently, but, in theory, *reference architectures* could also be seen as pseudo-oracles
 3. **Implicit oracles** are something we know based on our general software engineering knowledge; a *crash* of a system is clearly a bug, a *buffer overflow* is clearly a bug, and so forth and so on
 4. **Derived oracles** are something about right and wrong behavior based on external artifacts (such as requirements, protocols, standards, user feedback, runtime behavior, etc.)

Testing (3/6)

While true, do:

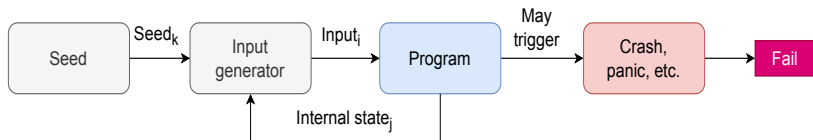
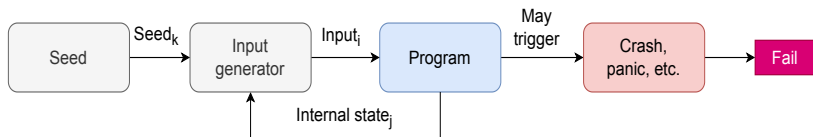


Figure 5: It Makes Sense But Why Is It Not Sufficient?

Testing (4/6)

While true, do:



If `get.time() > threshold`:

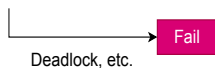
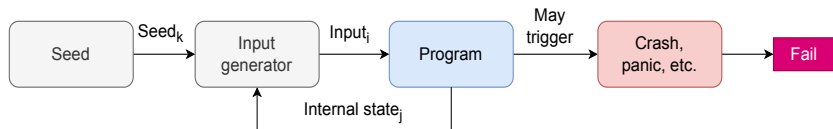


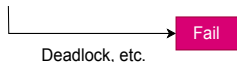
Figure 6: A Skeleton View to Fuzzing

Testing (5/6)

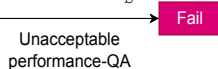
While true, do:



If `get.time() > threshold $_a$` :



If `get.load() > threshold $_b$` :



If `get.latency() > threshold $_c$` :

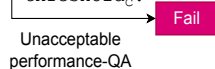


Figure 7: A Skeleton Fuzzer with Performance Checks

Testing (6/6)

- ▶ Code **sanitizers** are a good example of **dynamic testing**; these are tools that detects bugs in the form of undefined or otherwise suspicious behavior (e.g., deadlocks, memory leaks, etc.) via a compiler inserting instrumentation code at runtime
 - Many userland and kernel space fuzzers rely on these too; Adkins et al. (2020) (“the open access Google book from the first lecture”) present a good introduction

Kernel Address Sanitizer (KASAN)

English

Overview

Kernel Address Sanitizer (KASAN) is a dynamic memory safety error detector designed to find out-of-bounds and use-after-free bugs.

Figure 8: <https://docs.kernel.org/dev-tools/kasan.html>

Testability Tactics (1/2)

- ▶ From the previous figures we can deduce that testability improves if we can **control** a program's **input and the state**, which we should also **record** for debugging purposes
- ▶ Another set of tactics is about **limiting complexity**
 - Not only is a complex program (or a whole system) likely to contain many faults but complexity also decreases testability

Testability Tactics (2/2)

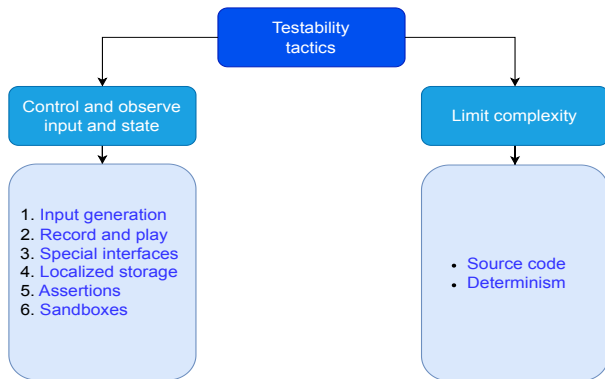


Figure 9: Tactics for Testability (from Bass et al. 2021)

Testability Tactics: Input and State (1/8)

- ▶ The **input generation** tactic involves many automated testing techniques beyond fuzzing, such as **mutation testing**
 - When doing conventional **unit testing** and alike, one should seek to design inputs to be sensible for verification, avoiding trivial cases but covering **boundary cases**
 - When operating with **standards or protocols**, seeking *100% coverage* can be recommend with respect to *what is explicitly said* in a given standard or a protocol

Testability Tactics: Input and State (2/8)

- ▶ The **record and play** tactic conveys the important message that test results should be **reproducible**
 - This requirement may seem deceptively simple when thinking about unit tests and their oracles, but reproducibility is one of the **grand challenges**, including with fuzzing and nowadays also emerging LLM-based testing
- ▶ When designing a testing framework, one should try to make it record all states for a given input
 - With distributed systems, one should also try to **localize storage** for keeping a state of a whole system

Testability Tactics: Input and State (3/8)

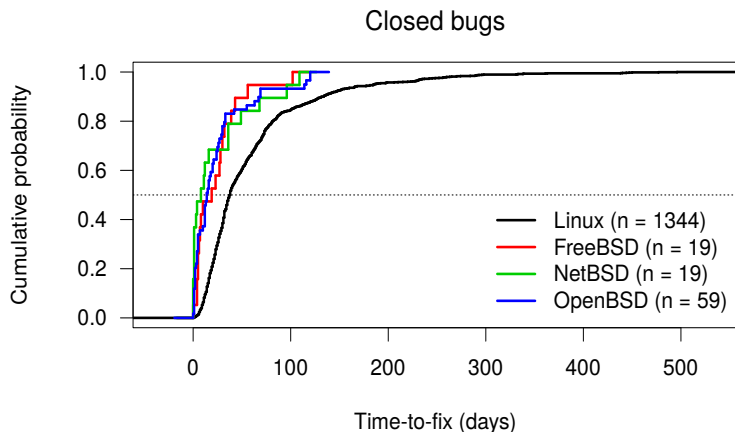


Figure 10: Effort Required to Fix Kernel Bugs Found via Continuous Automated Fuzzing (adopted from Ruohonen & Rindell 2019, Fig. 2)

Testability Tactics: Input and State (4/8)

- ▶ **Specialized interfaces** are often needed for testing internals
 - `get()` and `set()` for important variables, attributes, etc.
 - `report()` for states, `print_debug()` for verbose output, and `reset()` for restoring a pristine state
 - However, these also enlarge attack surfaces and may confuse others, and thus **conditional compilation** (or outsourcing to a specific opt-in component) is a good idea

```
#ifdef TESTING
    assert(system.report() == STATE_SHUTDOWN)

    if system.get_init() != 1:
        system.exit(INIT_FAILED)
#endif
```

Testability Tactics: Input and State (5/8)

- ▶ **Assertions** are a good way to verify runtime behavior
 - They also improve testability; they usually output debug information, fuzzers are very good at triggering them, etc.
- ▶ Typically, one uses them to verify (a) inputs (**preconditions**) and (b) outputs (**postconditions**), but sometimes it is useful to also assert (c) **intermediate** states
- ▶ They can be also extended for instance into:

```
ASSERT_FUTURE_STATE(10 seconds) == SYSTEM_SHUTDOWN  
ASSERT_PROBABILITY(RANDOM_FOREST, test_set) > 0.7
```


Testability Tactics: Input and State (6/8)

- ▶ A **sandbox** (for a lack of a better term) is a good idea; tests can be run in their own isolated environment that do not interfere with the actual system
- ▶ Increasingly – with DevOps and related practices soon discussed, tests are continuously run during automated integration and test deployments
 - **Operating systems** are a good example in a sense that an execution pipeline may be something alike:
for i times a day:
 build system → install system → run tests
where a successful build and an installation (into virtual machine typically) are prerequisites for running a test suite

Testability Tactics: Input and State (7/8)

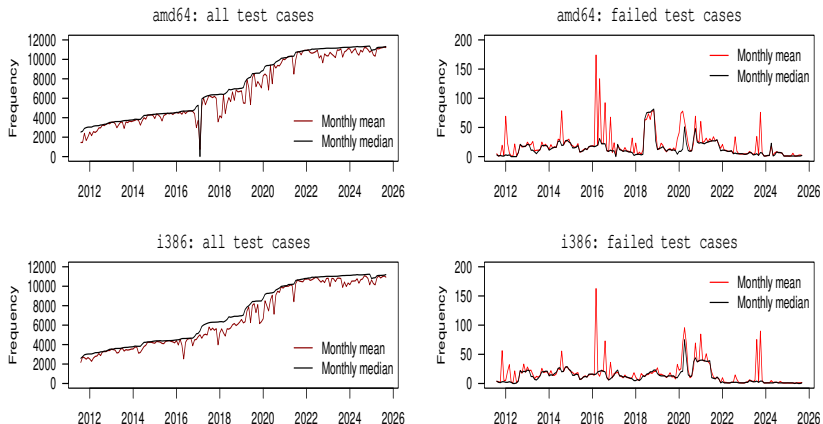


Figure 11: Test Cases and Failed Tests for Two Instruction Set Architectures in NetBSD (from Ruohonen & Tiwari 2025, Fig. 2)

Testability Tactics: Input and State (8/8)

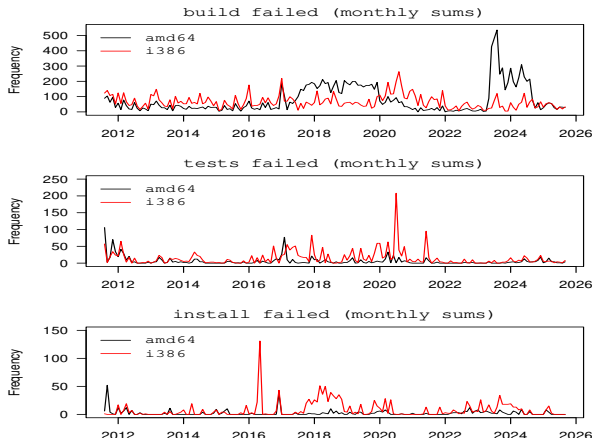


Figure 12: Monthly Sums of Build, Test Suite, and Installation Failures in NetBSD's Two Architectures (from Ruohonen & Tiwari 2025, Fig. 3)

Testability Tactics: Limit Complexity (1/3)

- ▶ In general, limiting complexity contains many of the same techniques that were discussed in the previous lectures; from encapsulation to reducing dependencies
- ▶ Therefore, what is more important to emphasize that **both systems and tests should ideally be deterministic**
 - However, they often are not for various reasons; there is a certain **stochastic element** present in large systems
 - For this reason, the NetBSD's test suite uses an oracle that has also flags for individual tests resembling cases such as “*expected failure due to a known extremely difficult bug*” and “*sometimes fails in ARM64 and MIPS for unknown reason*”

Testability Tactics: Limit Complexity (2/3)

- ▶ To control the stochastic element, including with respect to development processes, **regression testing** is commonly used
 - In the context of software testing, a **regression** is something that used to work but no longer works; hence, when fixing such a bug, it is a good idea to add a *regression test* to ensure the bug is noticed if it occurs again
- ▶ Regression tests are also about **documentation** about a system, which is also one way to limit complexity
 - Without regression tests, do you think the NetBSD developers would remember all bugs from the early 2010s?

Testability Tactics: Limit Complexity (3/3)

- ▶ If you know some parts are prone to regressions, it is a good idea to focus on those; knowing **what and where to test** is important for covering hot spots and not wasting resources

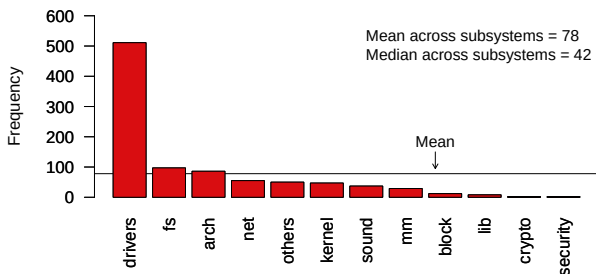


Figure 13: Regression Bugs in the Linux Kernel Across Its Subsystems Between 2021 and 2024 (from Ruohonen & Alami 2024, Fig. 3)

Testability Patterns: Dependency Injection

- ▶ **Dependency injection** means that a function or object receives other functions or objects it requires (instead of creating them itself)
- ▶ It helps (unit) testing because also test instances can be injected, and these can be further control and monitor a state
- ▶ Overhead and small complexity increases are the trade-offs

```
def __init__(self, x) -> None:  
    self.__x = x    # <-- dependency injection
```

Testability Patterns: Strategy Pattern

- ▶ When using the **strategy pattern**, an abstract method is present but the desired functionality can be chosen at runtime
- ▶ The pattern may simplify things, but readability may suffer

```
f(type, x):  
    switch(type):  
        case RANDOM_FOREST:  
            api::random_forest(x)  
        case SUPPORT_VECTOR_MACHINE:  
            api::svm(x)  
        ...  
    default:  
        assert(1 == 2)
```


Testability Patterns: Intercepting Filter Pattern

- ▶ The **intercepting filter pattern** injects pre- and post-processing filters to a function; these are executed before and after the function's own primary method is called

```
def __init__(self, pre, post):  
    self.__pre = pre  
    self.__post = post  
  
def __f(self, x):  
    if self.__pre != None:  
        self.__pre(x)  
    z = self.__g(x)  
    if self.__post != None:  
        self.__post(z)
```

Testability Patterns: Type Safety

- ▶ The **type safety** pattern means that one should use strongly typed languages or type annotations with weakly typed languages; these help at testing via static analyzers and can be used for instance in **taint analysis**

```
def f(x: int, y: int) -> int:
    #
    # The assert is here to remark that the
    # annotations are not enforced in Python.
    #
    assert type(x) == int
    return x + y
```

Testability Patterns: LLMs (1/7)

- ▶ The best practices are still being worked on, but again recall Feynman from the previous lecture and at least use a non-LLM tool (e.g., static analyzer) to check outputs

A vibe coder asked a LLM *“to generate automated tests to isolate functions but did not accept them at face value, noting: ‘I don’t think that’s quite right—it’s saying all the tests are passing, but it probably wrote the tests [so it can pass].’”* Those vibe coders relying heavily on LLMs requested *“summaries and assessments of terminal output and also prioritize issues: ‘Is this error important? Do we need to fix it?’”* Then, *“after reading the model’s suggested fix, they expressed skepticism: ‘Why would that be the issue?’”* (Chou et al. 2025, p. 15)

Testability Patterns: LLMs (2/7)

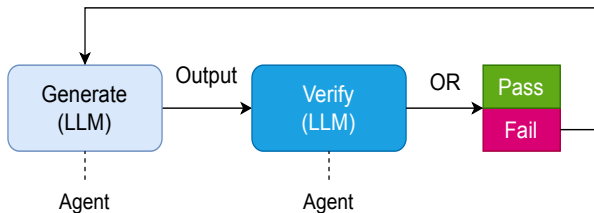


Figure 14: Not a Good Idea but Maybe in the Future?

Testability Patterns: LLMs (3/7)

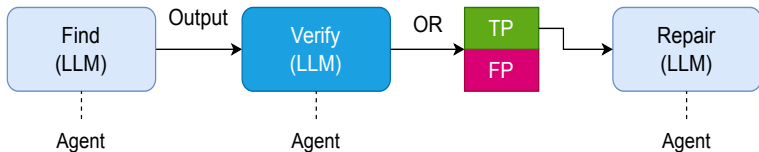


Figure 15: Another Bad Idea Currently but Maybe in the Future?

Testability Patterns: LLMs (4/7)

- ▶ At the moment, it would seem reasonable to argue that LLMs have indeed eased the producing of code, but, at the same time, **technical debt** is growing rapidly
- ▶ Recently, Bouzoukas (2026) used a specific term **verification debt** to underline that verifying and testing LLM output is a pressing challenge at the moment
 - Note also that **security debt** is a “sub-debt” here; LLMs are known to produce insecure code with a high probability, and this debt is not only yours because LLMs are increasingly used throughout software supply-chains, whether closed source or open source

Testability Patterns: LLMs (5/7)

 SIGN IN / UP

The  Register



SECURITY



AI has gotten good at finding bugs, not so good at swatting them

Discovery is getting cheaper. Validation and patching aren't

 [Thomas Claburn](#) Tue 24 Feb 2026 / 22:36 UTC

What good is finding a hole if you can't fix it? Anthropic last week talked up Claude Code's improved ability to find software vulnerabilities and propose patches. But security researchers say that's not enough.

The AI biz describes the research preview capability, dubbed [Claude Code Security](#), as a way for security teams to find and fix flaws they might otherwise have missed.

"This is a pivotal time for cybersecurity," the company [said](#). "We expect that a significant share of the world's code will be scanned by AI in the near future, given how effective models have become at finding long-hidden bugs and security issues."

To highlight Claude Code Security's bug hunting potential, the company pointed to how its red team had used Claude Opus 4.6 to find [over 500 vulnerabilities](#) in production open-source codebases."

Figure 16: https://www.theregister.com/2026/02/24/ai_finding_bugs/

Testability Patterns: LLMs (6/7)

OSS-Fuzz: Continuous Fuzzing for Open Source Software

[Fuzz testing](#) is a well-known technique for uncovering programming errors in software. Many of these detectable errors, like [buffer overflow](#), can have serious security implications. Google has found [thousands](#) of security vulnerabilities and stability bugs by deploying [guided in-process fuzzing of Chrome components](#), and we now want to share that service with the open source community.

In cooperation with the [Core Infrastructure Initiative](#) and the [OpenSSF](#), OSS-Fuzz aims to make common open source software more secure and stable by combining modern fuzzing techniques with scalable, distributed execution. Projects that do not qualify for OSS-Fuzz (e.g. closed source) can run their own instances of [ClusterFuzz](#) or [ClusterFuzzLite](#).

Trophies

As of May 2025, OSS-Fuzz has helped identify and fix over 13,000 vulnerabilities and 50,000 bugs across [1,000](#) projects.

Figure 17: <https://github.com/google/oss-fuzz>

Testability Patterns: LLMs (7/7)

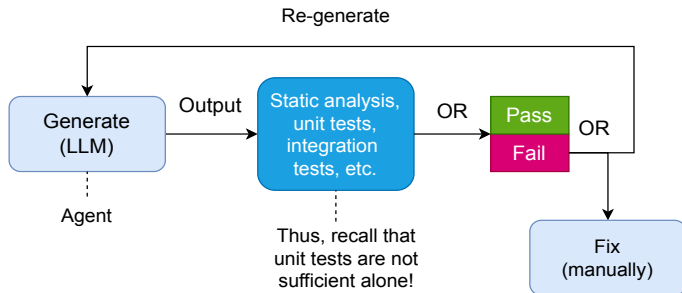


Figure 18: To Be on the Safe Side, I Would Recommend Keeping Validation Separate from LLMs Also When “Vibe Coding”

Testability Patterns: There Is Much More! (1/2)

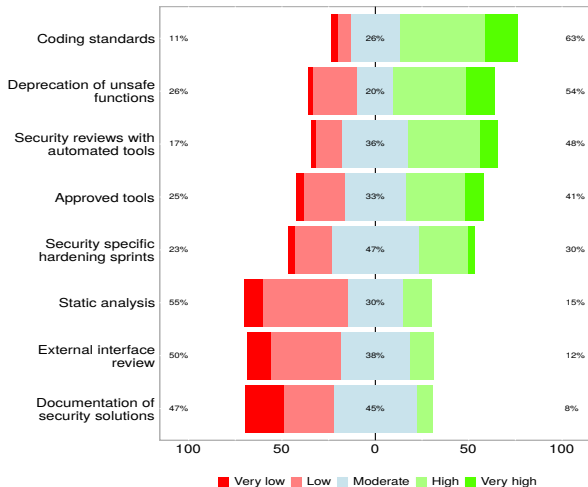


Figure 19: Rindell et al. (2021, Fig. 6)

Question

Which Ones You Think a LLM Could Do?

Testability Patterns: There Is Much More! (2/2)

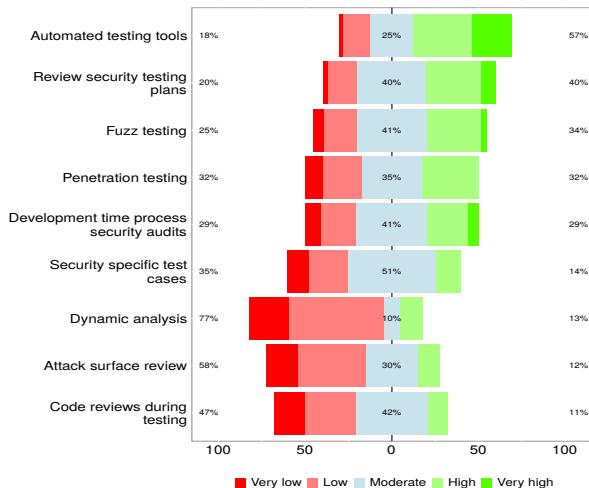


Figure 20: Rindell et al. (2021, Fig. 8)

Question

Which Ones You Think a LLM Could Do?

Deployability

- ▶ Traditionally — and, depending on a domain, sometimes still today, **release engineering** was a cumbersome process; a release was planned and tested for a long time
- ▶ From the late 2000s onward, so-called **continuous software engineering** emerged, among them **continuous integration** and **continuous delivery**
 - A concept of **rapid releases** is also sometimes used
- ▶ Recall the definition; it should be possible to deploy a system with **predictable and acceptable amount** of time and effort
 - This point underlines what was said earlier about QAs as measurable or even falsifiable attributes
 - Besides reduced time and effort, the continuous software engineering practices have improved also predictability

Continuous Integration and Continuous Deployment

- ▶ **Continuous integration** means that a system being developed is continuously (e.g., daily) built and tested, such as that a working version is always available
- ▶ If a system is further deployed into production on a regular basis (e.g., monthly), a further concept of **continuous deployment** is often used
- ▶ The notion of **DevOps** covers both (and possibly more)
- ▶ These continuous practices require separating different environments each other; e.g., “*development environment* → *integration environment* → *staging environment* → *production environment*”

Deployment Pipelines (1/4)

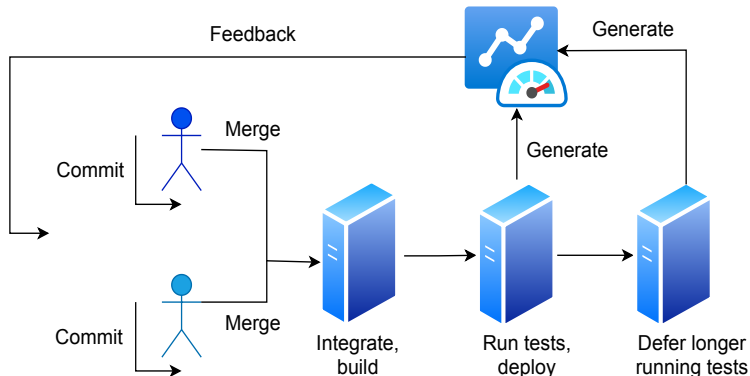


Figure 21: An Illustration of Continuous Integration and Deployment (adopted from Duvall et al. 2007, Fig. 4-6)

Deployment Pipelines (2/4)

- ▶ As a QA, deployability with a pipeline should be measurable
- ▶ Bass et al. (2021) suggest to consider the following:
 1. **Cycle time:** how long does it take to progress through a given deployment pipeline (i.e., from the development to the deployment environment)?
 2. **Traceability:** is it possible to trace forward (from the development to the deployment environment) and backwards (from the deployment to the development environment)?
 3. **Repeatability:** is the whole process reproducible; does it always give the same outcome with the same artifacts?
 - Easier said than done even with respect to builds alone!

Deployment Pipelines (3/4)



Figure 22: <https://reproducible-builds.org/>

Deployment Pipelines (4/4)

- ▶ There are alternatives to GitHub (and everything else for that matter if you bother to look around for a moment)



Figure 23: <https://about.gitea.com/>

Deployability Scenarios

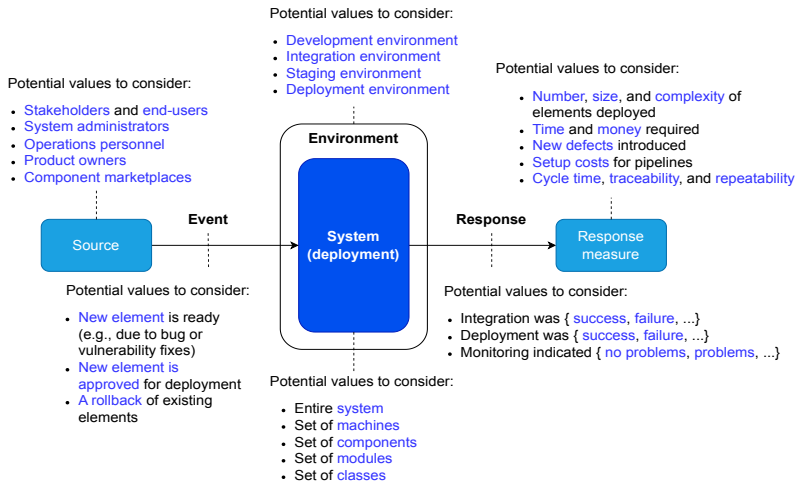


Figure 24: Adopted and modified from Bass et al. (2021)

Deployability Tactics

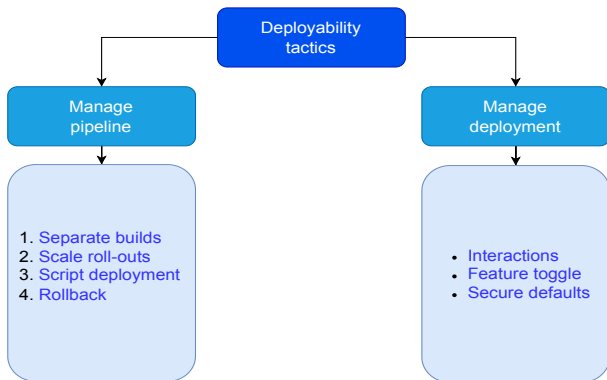


Figure 25: Tactics for Deployability

Deployability Tactics: Manage Pipeline (1/4)

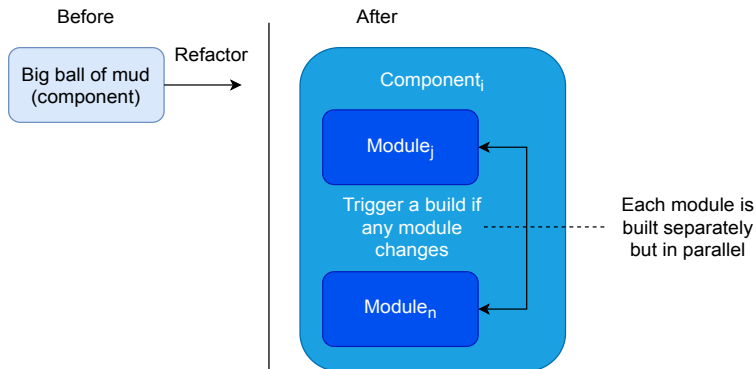


Figure 26: Separate Builds (adopted from Duvall et al. 2007, Fig. 4-7)

Deployability Tactics: Manage Pipeline (2/4)

- ▶ When operating with large systems, it is a good idea to **scale deploying incrementally to subsets** of end-user population
 - This practice is also known as **canary testing**
- ▶ Sheehy & Reed (2025) recommend having a solid plan for **verifying deployment failures** as well as for **measuring and documenting** these with predefined **health metrics**
 - Only when these indicate no problems for a subset of end-users (or endpoints), a **full roll-out** should be done
 - With particularly large systems, it may be reasonable to define **multiple subsets** according to some criteria (e.g., different geographic locations, different configurations, etc.)

Deployability Tactics: Manage Pipeline (3/4)

- ▶ For the scaling to be possible, deploying should be **scripted**
 - Although some manual configuration is still typically required, **automate as much as possible**
- ▶ When health metrics indicate problems, it is important to have a **rollback** functionality; it should be possible to revert back to the previously deployed version of a system
 - This functionality correlates with the **traceability** and repeatability requirements; for a rollback to be possible, all configurations and variability characteristics should be tracked

Deployability Tactics: Manage Pipeline (4/4)

FreeBSD Manual Pages

[home](#) | [help](#)

ZFS-ROLLBACK(8) System Manager's Manual *ZFS-ROLLBACK(8)*

NAME
 `zfs-rollback -- roll` ZFS dataset back to snapshot

SYNOPSIS
 `zfs rollback [-Rfr] snapshot`

DESCRIPTION
 When a dataset is rolled back, all data that has changed since the snapshot is discarded, and the dataset reverts to the state at the time of the snapshot. By default, the command refuses to roll back to a snapshot other than the most recent one. In order to do so, all intermediate snapshots and bookmarks must be destroyed by specifying the `-r` option.

Figure 27: <https://man.freebsd.org/cgi/man.cgi?query=zfs-rollback>

Deployability Tactics: Manage Deployment (1/4)

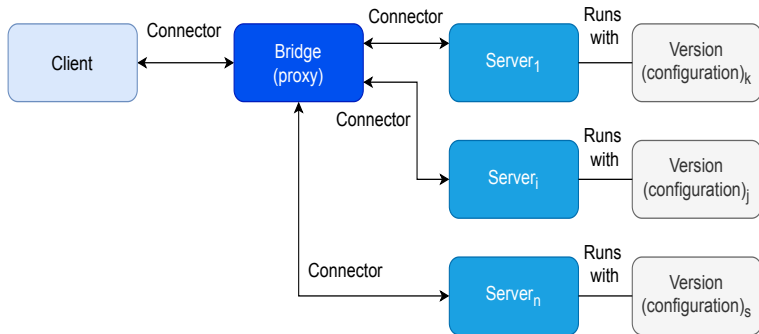


Figure 28: Multiple Servers with Different Versions (or Configurations)

Deployability Tactics: Manage Deployment (2/4)

- ▶ For verification and measurement purposes, it may be a good idea to have a **feature toggle** functionality that allows enabling or disabling features at runtime
 - A “**kill-switch**” is an example
- ▶ However, deployments into production should always come with **safe and secure defaults**, meaning that *functional features should be off by default*, whereas *security features should be on by default* (Ruohonen 2025)

Deployability Tactics: Manage Deployment (3/4)

GitOps

Since its inception in 2017 by [Weaveworks](#), GitOps has caused quite some fuss on Twitter and KubeCon. This site aggregates the essence of GitOps to help clear up the confusion about the topic.



We've written a short book on GitOps. Get it as a [free PDF](#) or [ePub](#), or for a small fee as a [Kindle eBook](#) or [paperback](#).

Figure 29: <https://www.gitops.tech/>

Deployability Tactics: Manage Deployment (4/4)

- ▶ The continuous software engineering practices, including DevOps and GitOps, convey a general message that one should **automate as much as possible**, and doing continuous monitoring, **log-analysis**, and analytics once automated
 - Hence, there are also further related notions such as *“pipeline as code”* and *“infrastructure as code”*
- ▶ Recently, as outlined by Wei et al. (2025), further concepts have emerged, such as *“compliance as code”*, *“security as code”*, *“configuration as code”*, and *“network as code”*
- ▶ Do you think **“everything as code”** might be the future?

Deployability Patterns: Microservices (1/2)

- ▶ **Microservices** are a good example as a deployment pattern
 - Though, to raise generalizability, in what follows a term **server** is sometimes used in place of a service
- ▶ The **benefits** include:
 1. **Distributed development and deployment:** in theory, teams can deploy their own services without a need to consider programming language choices or software evolution (insofar as required and provided interface remain compatible)
 - ⇒ Reduced integration errors due to interoperability issues and shortened time-to-market
 2. **Scaling** is easy at least on paper because new service instances can be added dynamically and also supply can be matched to demand dynamically

Deployability Patterns: Microservices (2/2)

- ▶ The **drawbacks** include:
 1. **Overhead** is larger than with in-memory connectors
 2. **Synchronization** can be challenging for complex transactions
 3. **Freedom of teams** to choose programming languages and other technologies increase the complexity of a whole system
 4. **Responsibilities** and **level of granularity** of services can be challenging to design
 5. **Catalogs** (incl. SBOMs) of services cause overhead

Deployability Patterns: Rolling Upgrades (1/8)

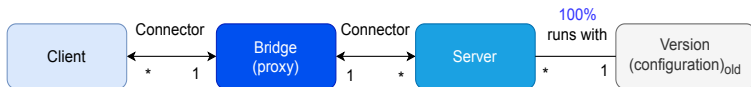


Figure 30: Rolling Upgrades #1 (Bass et al. 2021)

Deployability Patterns: Rolling Upgrades (2/8)

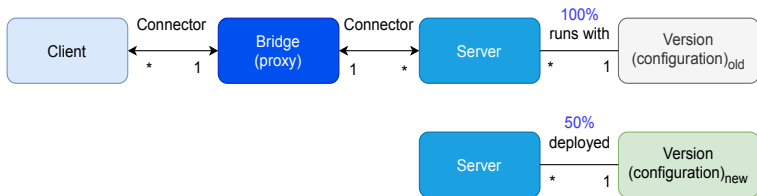


Figure 31: Rolling Upgrades #2 (Bass et al. 2021)

Deployability Patterns: Rolling Upgrades (3/8)

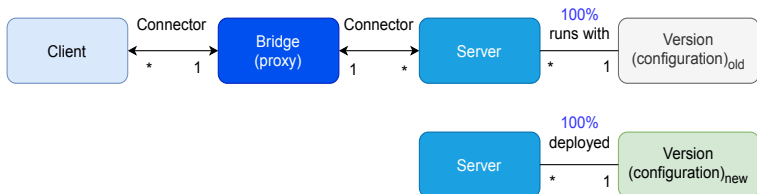


Figure 32: Rolling Upgrades #3 (Bass et al. 2021)

Deployability Patterns: Rolling Upgrades (4/8)

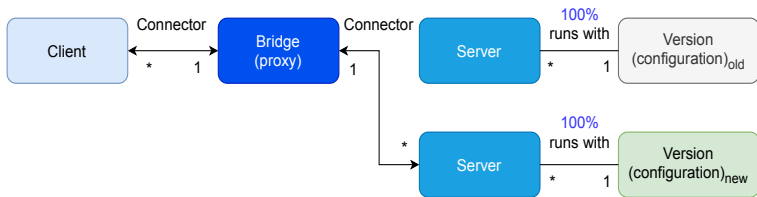


Figure 33: Rolling Upgrades #4 (Bass et al. 2021)

Deployability Patterns: Rolling Upgrades (5/8)

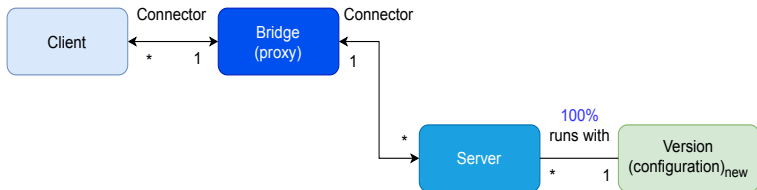


Figure 34: Rolling Upgrades #5 (Bass et al. 2021)

Deployability Patterns: Rolling Upgrades (6/8)

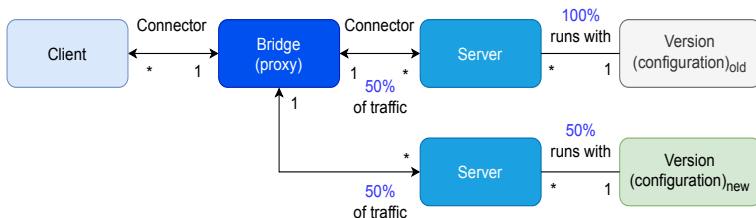


Figure 35: Alternative Rolling Upgrades #1 (Bass et al. 2021)

Deployability Patterns: Rolling Upgrades (7/8)

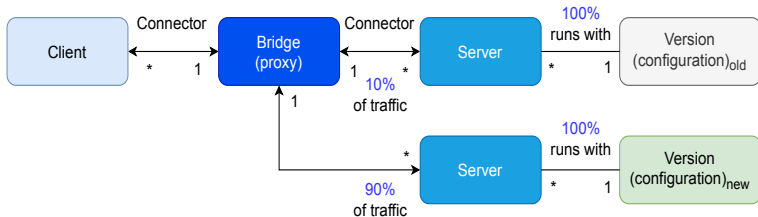


Figure 36: Alternative Rolling Upgrades #2 (Bass et al. 2021)

Deployability Patterns: Rolling Upgrades (8/8)

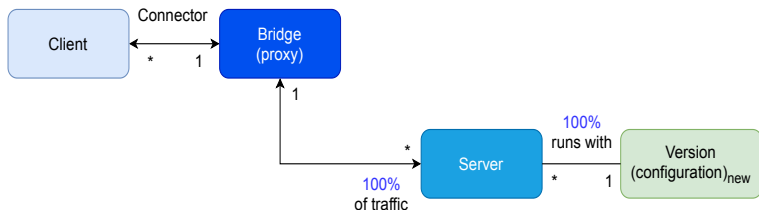


Figure 37: Alternative Rolling Upgrades #3 (Bass et al. 2021)

Deployability Patterns: A/B Testing (1/2)

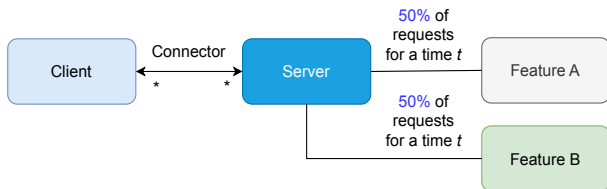


Figure 38: An Illustration of A/B Testing #1

Deployability Patterns: A/B Testing (2/2)

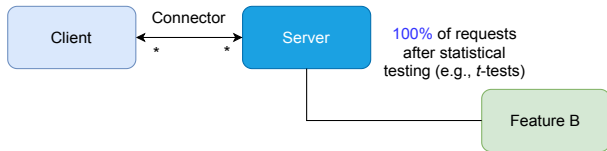


Figure 39: An Illustration of A/B Testing #2

Deployability Patterns: Legal Requirements (1/2)

- ▶ The new **Cyber Resilience Act** (CRA) of the European Union, which will be enforced from 2027 onward, requires providing security updates for a product's whole life cycle
- ▶ In addition, it says that these should be done **automatically** whenever possible and wanted by a customer
- ▶ Furthermore, it mandates the good practice that, whenever possible, security updates should be **separated from functional updates** that bring new features

Deployability Patterns: Legal Requirements (2/2)

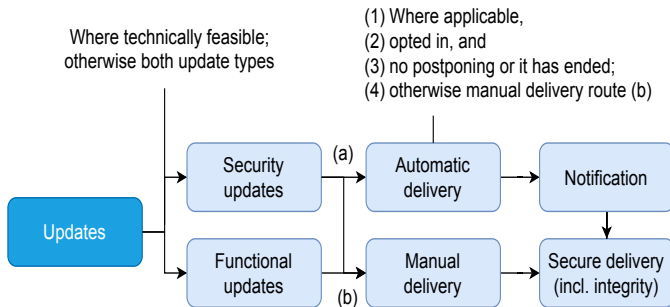


Figure 40: The CRA's Requirements for Security Updates (adopted from Ruohonen et al. 2025, Fig. 5)

Deployability Patterns: Release Engineering (1/2)

- ▶ The DevOps practices and particularly continuous deployment may lead to a **false conclusion or false impression** that everything would be continuously deployed everywhere
- ▶ Therefore, recall what was said in the earlier lecture about product lines and branching of releases
- ▶ In many cases, multiple releases may operate in parallel, and these may have varying support periods
 - Operating systems are good example; often there is a so-called **long-term support** (LTS) version and many versions with shorter support periods
 - Typically, the LTS versions do not see new features

Deployability Patterns: Release Engineering (2/2)

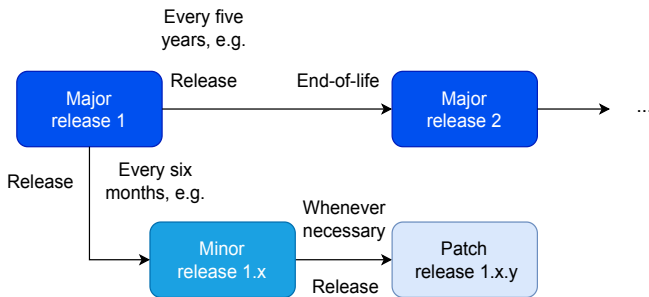


Figure 41: The Semantic Versioning Scheme Revisited

Deployability Patterns: Deployment Testing (1/4)



[About](#) [Conferences](#) [Publications](#) [Membership](#) [Students](#)

 [Join the conversation](#)
[Back to ;login: Online](#)

[;login:](#)

Running Disaster Recovery Plan Tabletop Exercises

March 26, 2025

Figure 42: <https://www.usenix.org/publications/loginonline/running-disaster-recovery-plan-tabletop-exercises>

PRINCIPLES OF CHAOS ENGINEERING

Last Update: 2019 March ([changes](#))

Chaos Engineering is the discipline of experimenting on a system in order to build confidence in the system's capability to withstand turbulent conditions in production.

Figure 43: <https://principlesofchaos.org/>

Deployability Patterns: Deployment Testing (3/4)

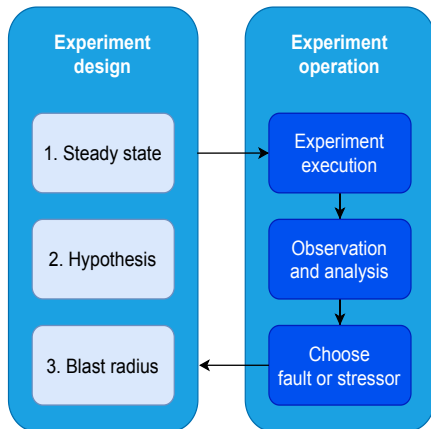
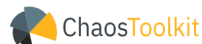


Figure 44: Setup for Chaos Engineering and Tabletop Experiments

Deployability Patterns: Deployment Testing (4/4)



Search



chaostoolkit

The chaos engineering toolkit for developers

Explore and test your systems to discover their weaknesses.

Get started

View on GitHub

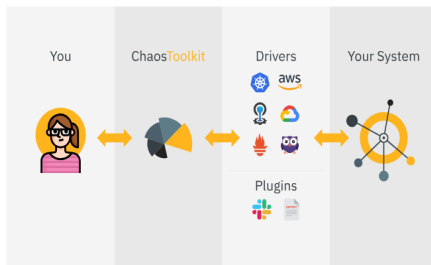


Figure 45: <https://chaostoolkit.org/>

Questions?

Case #4.A (choose either this case or the case B)

- ▶ Investigate the *syzkaller/syzbot* (as also done by Ruohonen & Rindell 2019) used to continuously fuzz the Linux kernel:
<https://syzkaller.appspot.com>
- ▶ Can you say anything about the **invalid cases**?
- ▶ What can you say by investigating the **graphing functionality** provided on the dashboard, including with respect to fuzzing, open bugs, and coverage?
 - Presenting graphs would be nice but not mandatory
- ▶ Are all **subsystems** tested equally well?
- ▶ Has the kernel's "**health**" been stable in overall?
- ▶ Any other insights you might want to share?

Case #4.B (choose either this case or the case A)

- ▶ **CrowdStrike** had a major incident in July 2024 and then **Cloudflare** in December 2025
- ▶ Read Internet sources about these incidents with respect to:
 - What were the **causes**?
 - What were the **effects**?
 - What do the incidents say about **testability**?
 - How about **deployability**?
 - What can be **learned** from the incidents?

Thank you

Questions?

Additional References (1/3)

1. Boehm, B. W. (1984): Verifying and Validating Software Requirements and Design Specifications. IEEE Software (1)(1), pp. 75–88, <https://doi.org/10.1109/MS.1984.233702>
2. Ammann, P. & Offutt, J. (2008): Introduction to Software Testing. Cambridge University Press, <http://www.cambridge.org/9780521880381>
3. Barr, E. T. & Harman, M. & McMinn, P. & Shahbaz, M. & Yoo, S. (2015): The Oracle Problem in Software Testing: A Survey. IEEE Transactions on Software Engineering 41(5), pp. 507–525, <https://doi.org/10.1109/TSE.2014.2372785>
4. Ruohonen, J. & Rindell, K (2019): Empirical Notes on the Interaction Between Continuous Kernel Fuzzing and Development. International Symposium on Software Reliability Engineering Workshops (ISSREW 2019), IEEE, pp. 276–281, <https://arxiv.org/pdf/1909.02441>
5. Ruohonen, J. & Tiwari, A. (2025): Empirical Derivations from an Evolving Test Suite. A pre-print under review, <https://arxiv.org/pdf/2511.00915>

Additional References (2/3)

1. Ruohonen, J. & Alami, A. (2024): Fast Fixes and Faulty Drivers: An Empirical Analysis of Regression Bug Fixing Times in the Linux Kernel.
<https://arxiv.org/pdf/2411.02091>
2. Chou, Y-H. et al. (2025): Building Software by Rolling the Dice: A Qualitative Study of Vibe Coding. A pre-print, <https://arxiv.org/pdf/2512.22418>
3. Bouzoukas, K. (2026): Verification Debt: When Generative AI Speeds Change Faster Than Proof. *Communications of the ACM* (Blog),
<https://cacm.acm.org/blogcacm/verification-debt-when-generative-ai-speeds-change-faster-than-proof/>,
4. Rindell, K. et al. (2021): Security in Agile Software Development: A Practitioner Survey. *Information and Software Technology* 131, p. 106488,
<https://doi.org/10.1016/j.infsof.2020.106488>
5. Duvall, P. M. & Matyas, S. & Glover, A. (2007): Continuous Integration. Addison-Wesley, <https://martinfowler.com/books/duvall.html>

Additional References (3/3)

1. Sheehy, J. & Reed, J. (2025): Moving Faster by Not Breaking Things —Or How Not to Blow Up the Internet. ACM Queue 23(4), <https://queue.acm.org/detail.cfm?id=3762988>
2. Ruohonen, J. (2025): SoK: The Design Paradigm of Safe and Secure Defaults. Journal of Information Security and Applications 90, pp. 103989, <https://doi.org/10.1016/j.jisa.2025.103989>
3. Ruohonen, J. & Hjerpe, K. & Kang, E-Y. (2025): A Mapping Analysis of Requirements Between the CRA and the GDPR. IEEE 33rd International Requirements Engineering Conference Workshops (REW 2025), IEEE, pp. 215–222, <https://arxiv.org/pdf/2503.01816>
4. Wei, H. & Madhavji, N. & Steinbacher, J. (2025): Understanding Everything as Code: A Taxonomy and Conceptual Model. <https://arxiv.org/pdf/2507.05100>